OSI Layer and Elastic Load Balancer

Class no:19

What is a Load Balancer?

A **load balancer** is a networking device or service that acts as an intermediary between clients and servers. Its primary function is to **distribute incoming network traffic across multiple servers** to ensure no single server is overwhelmed, improving:

- **Availability**: Keeps services running even if some servers fail.
- **Scalability**: Handles increased traffic by routing requests to multiple targets.
- **Performance**: Reduces response times by evenly distributing workloads.
- **Reliability**: Monitors server health and routes traffic only to healthy targets.

How it works:

1. Receives traffic on defined **ports and protocols** (HTTP, HTTPS, TCP, UDP).
2. Selects a server from a **target group** using algorithms like round-robin, least connections, or source IP hash.
3. Forwards requests to the target and sends the processed response back to the client.
4. Continuously performs **health checks** on servers to ensure traffic only reaches available targets.

OSI Model (Open Systems Interconnection)

The OSI model is a **conceptual framework for networked systems**, dividing telecommunications functions into seven layers:

1. **Physical Layer (L1)**: Transmission of raw binary data over a medium.
2. **Data Link Layer (L2)**: Error detection, MAC addressing, frame delivery.
3. **Network Layer (L3)**: Routing and IP addressing.
4. **Transport Layer (L4)**: Reliable data transfer, TCP/UDP communication, session multiplexing.
5. **Session Layer (L5)**: Managing and maintaining application sessions.
6. **Presentation Layer (L6)**: Data translation, encryption/decryption, compression.
7. **Application Layer (L7)**: Interface for end-user applications; HTTP, HTTPS, DNS, etc.

Load Balancing across OSI Layers:

- **Layer 3 (Network Layer)**: Routes traffic based on IP addresses. Minimal overhead, high throughput.
- **Layer 4 (Transport Layer)**: Uses TCP/UDP headers for routing. Efficient for session management; limited a pplication awareness.
- **Layer 7 (Application Layer)**: Routes based on content (HTTP headers, URLs, cookies). Intelligent, granular routing, supports advanced features like SSL termination, WebSockets, and content-based routing.

Types of Load Balancers in AWS and Their OSI Layer Differences

AWS provides **four primary load balancers**:

Type	**OSI Layer**	**Protocols**
Application Load Balancer (ALB)	7 (Application Layer)	HTTP, HTTPS, WebSocket, gRPC
Network Load Balancer (NLB)	4 (Transport Layer)	TCP, UDP, TLS
Gateway Load Balancer (GWLB)	3 & (appliance-aware)	IP Packets (transparent forwarding)
Classic Load Balancer (CLB)	4 & 7	TCP, HTTP, HTTPS

Key Differences:

1. **OSI Layers**: ALB (L7) > intelligent content-based routing; NLB (L4) > connection-based, fast; GWLB (L3) > virtual appliance scaling; CLB (L4/L7) > legacy, fewer features.
2. **Routing Logic**: ALB routes by URL, headers, or queries; NLB routes by IP/port; GWLB forwards packets to appliances; CLB uses basic load distribution.
3. **Features & Use Cases**: ALB is best for web apps with complex routing; NLB for low-latency high-throughput traffic; GWLB for security appliances; CLB for simpler or legacy workloads.

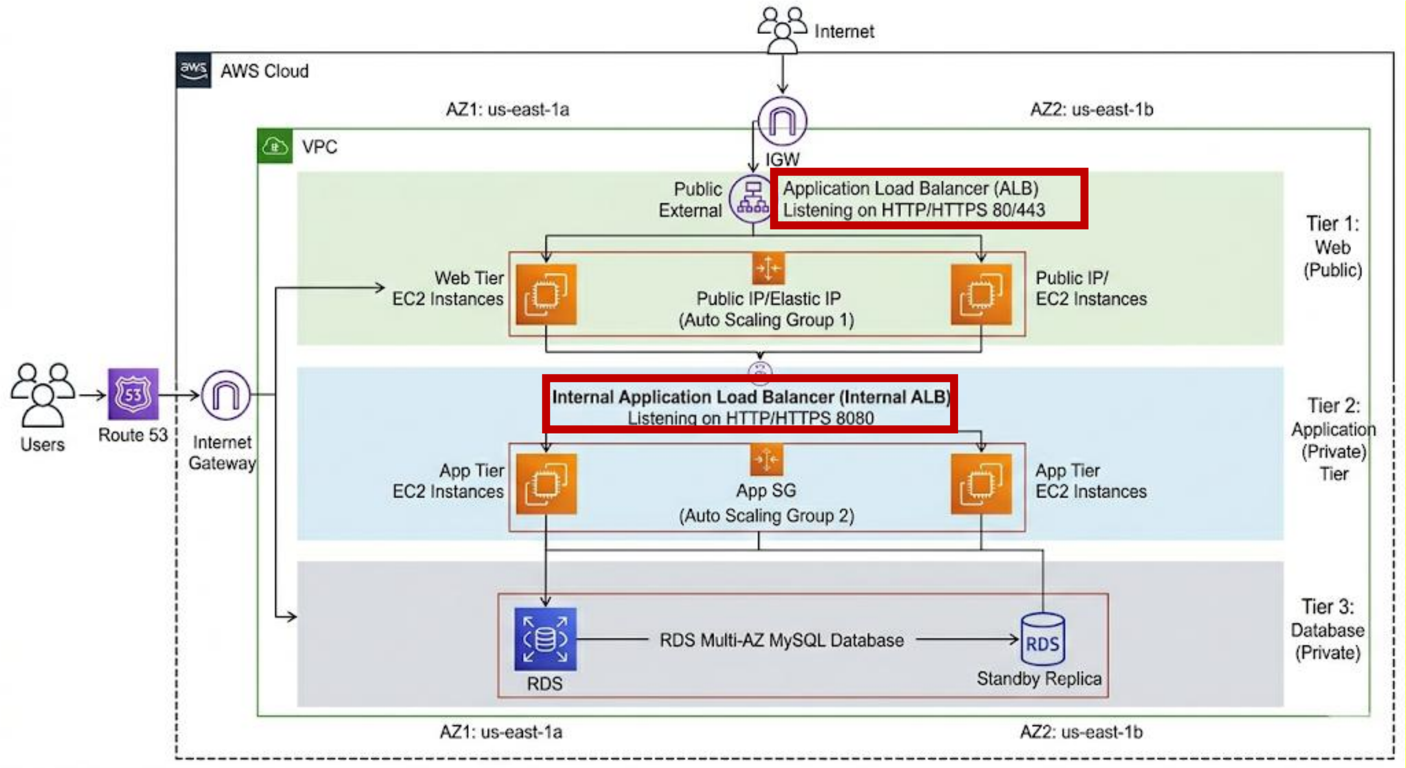
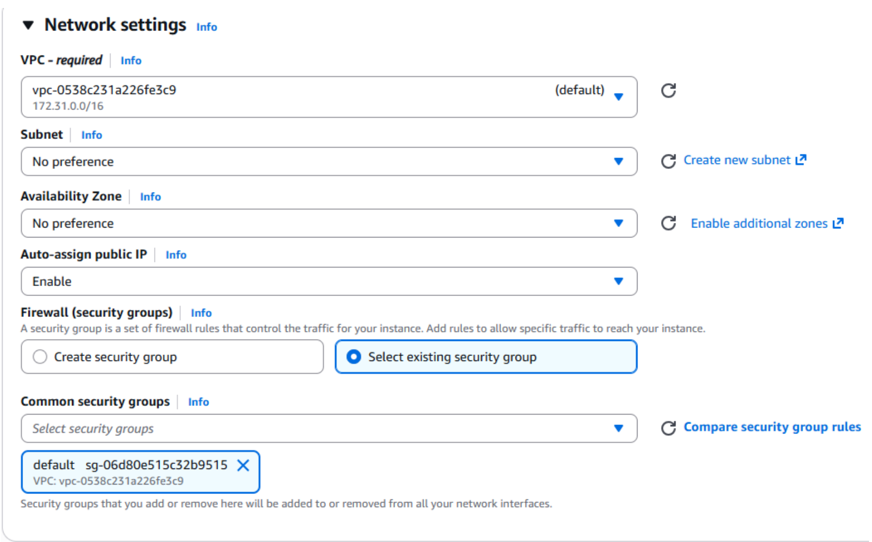


Fig. Architecture of a three-tier application with EC2 and Load balancers

Hands-on: Creating a load balancer with two Ec2 instances

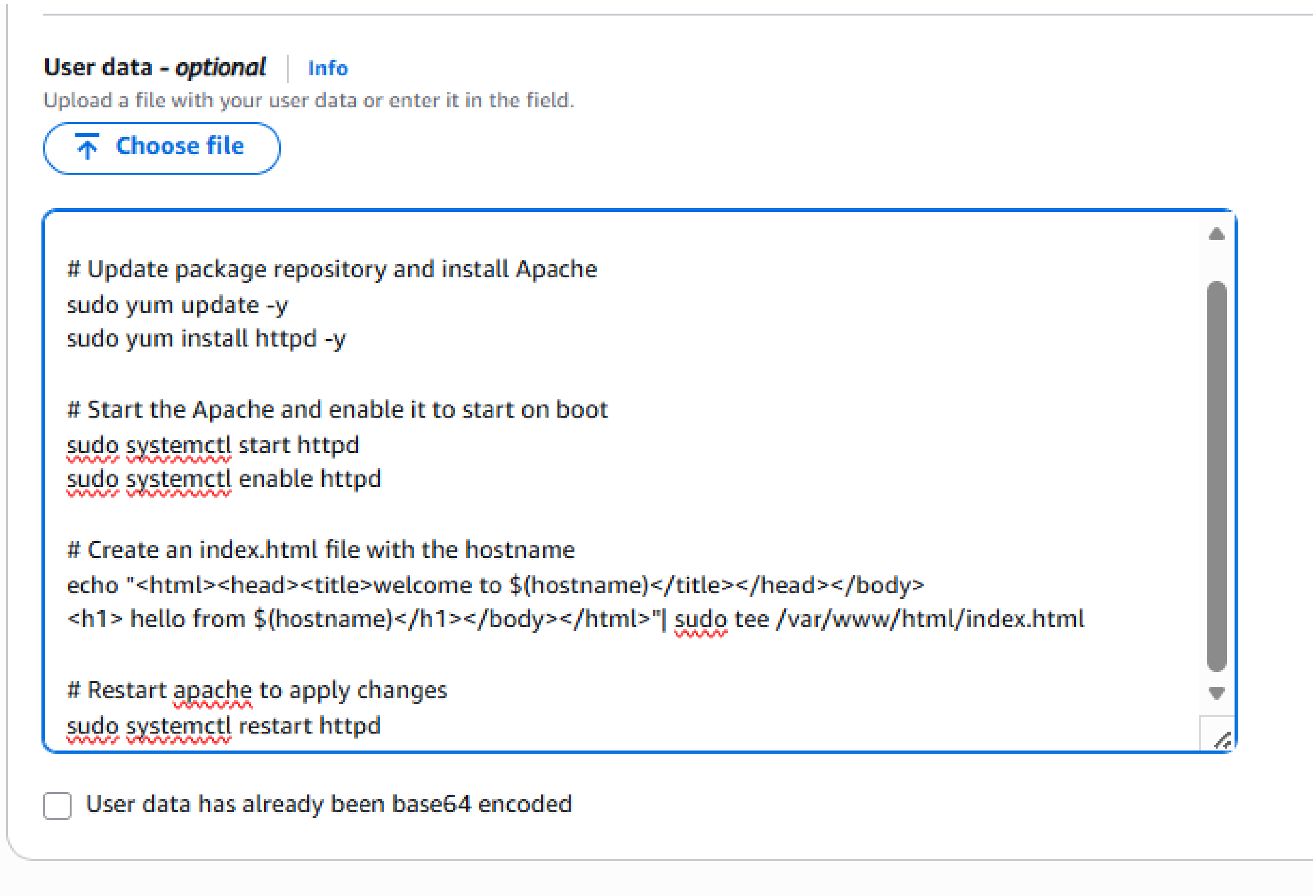
Step no:1 Create & Launch Two EC2 Instances

- Name: my-LB-instance1 & my-LB-instance2
- Use Amazon Linux 2 AMI.
- Security group → select existing group → default.



- In the **Advanced Details → User Data**,

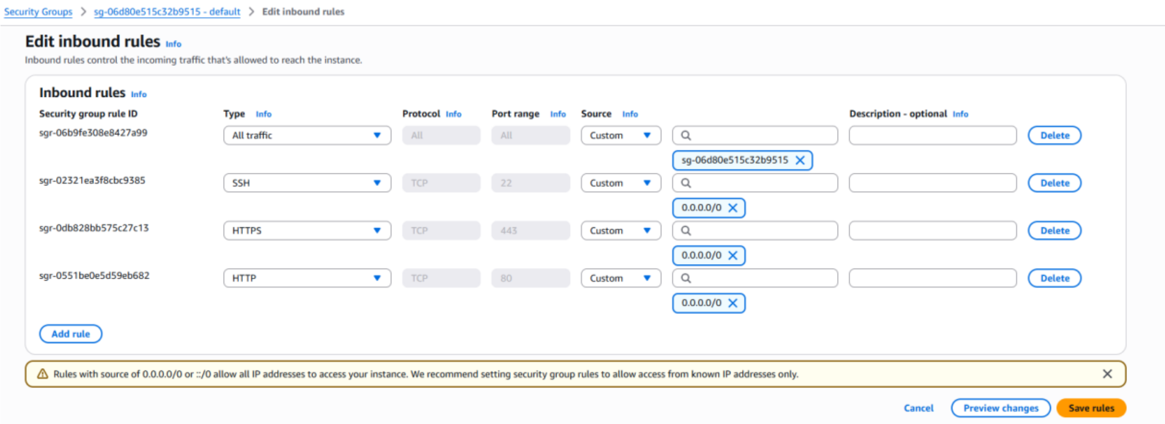
  paste your script → launch wo instances.

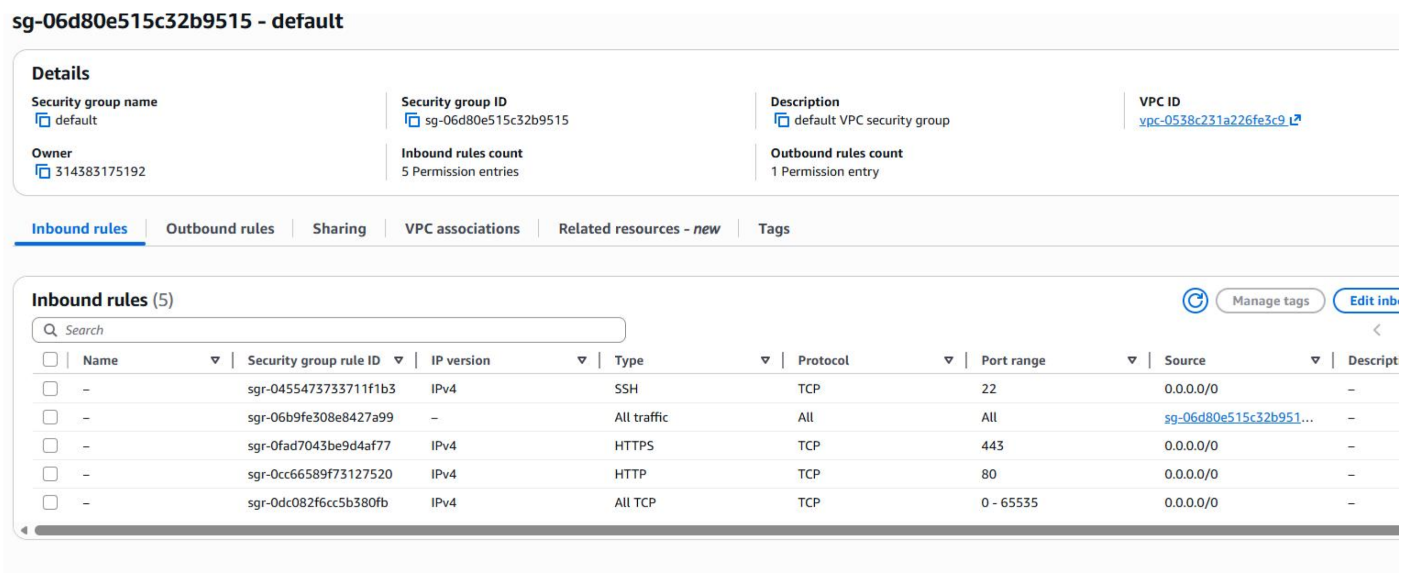


<u>Step no:02</u> Open Security Group Ports

Ensure both EC2 instances' security groups allow:

- **Inbound HTTP (port 80)** from anywhere (0.0.0.0/0).
- **Inbound SSH (port 22)** for admin access.
- **All TCP (0-65535)** It will enable (SSH as well as HTTP and HTTPS. They all are part of the TCP.)





<u>Step no:03</u> Verify Apache Installation in our instances by,

- After launch, connect via browser: http://<Public_IP_of_Instance> in a private window.

Public ip of my-LB-instance1:

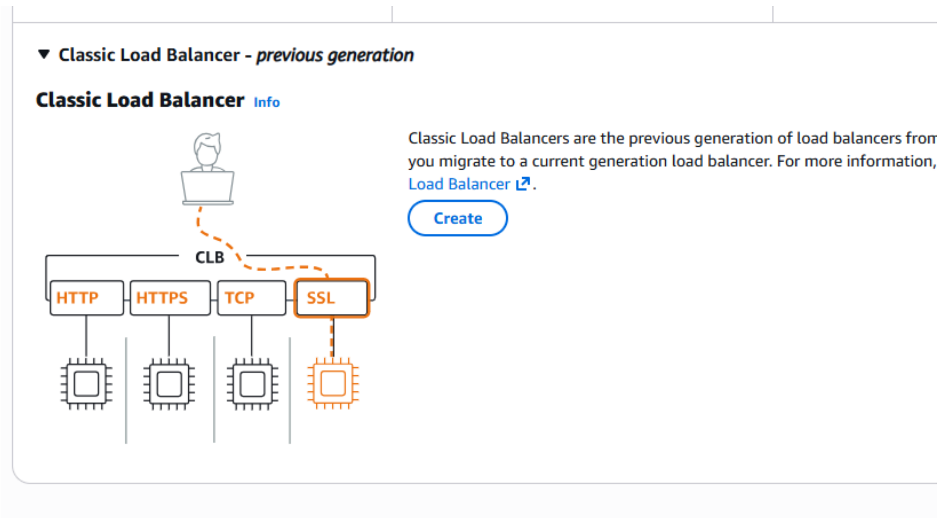
You should see: *hello from <hostname>*. (hostname is DNS+some ip address)

Public ip of my-LB-instance2:

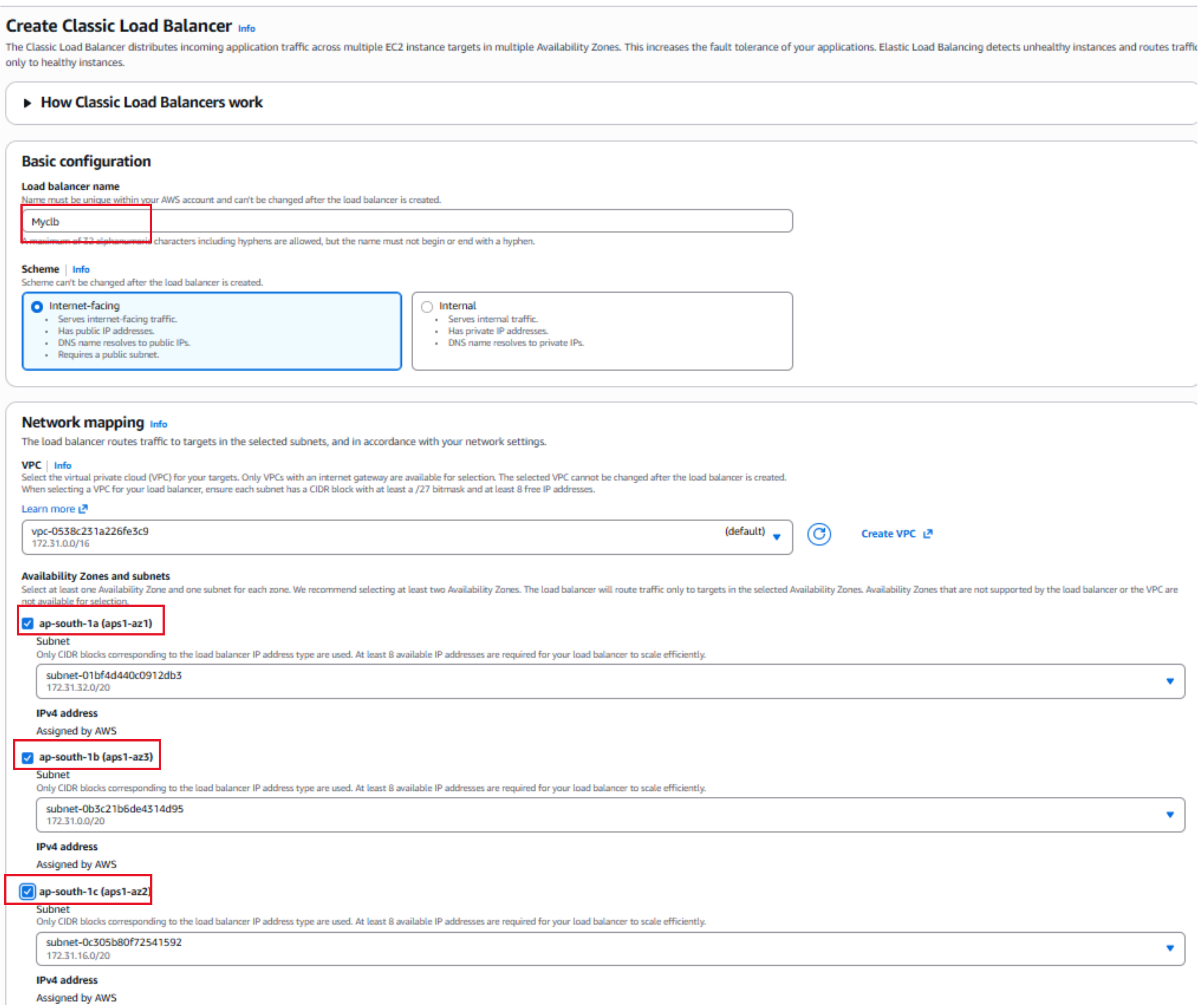
You should see: *hello from <hostname>*. (hostname is DNS+some ip address)

<u>Step no:4</u> Create Classic Load Balancer

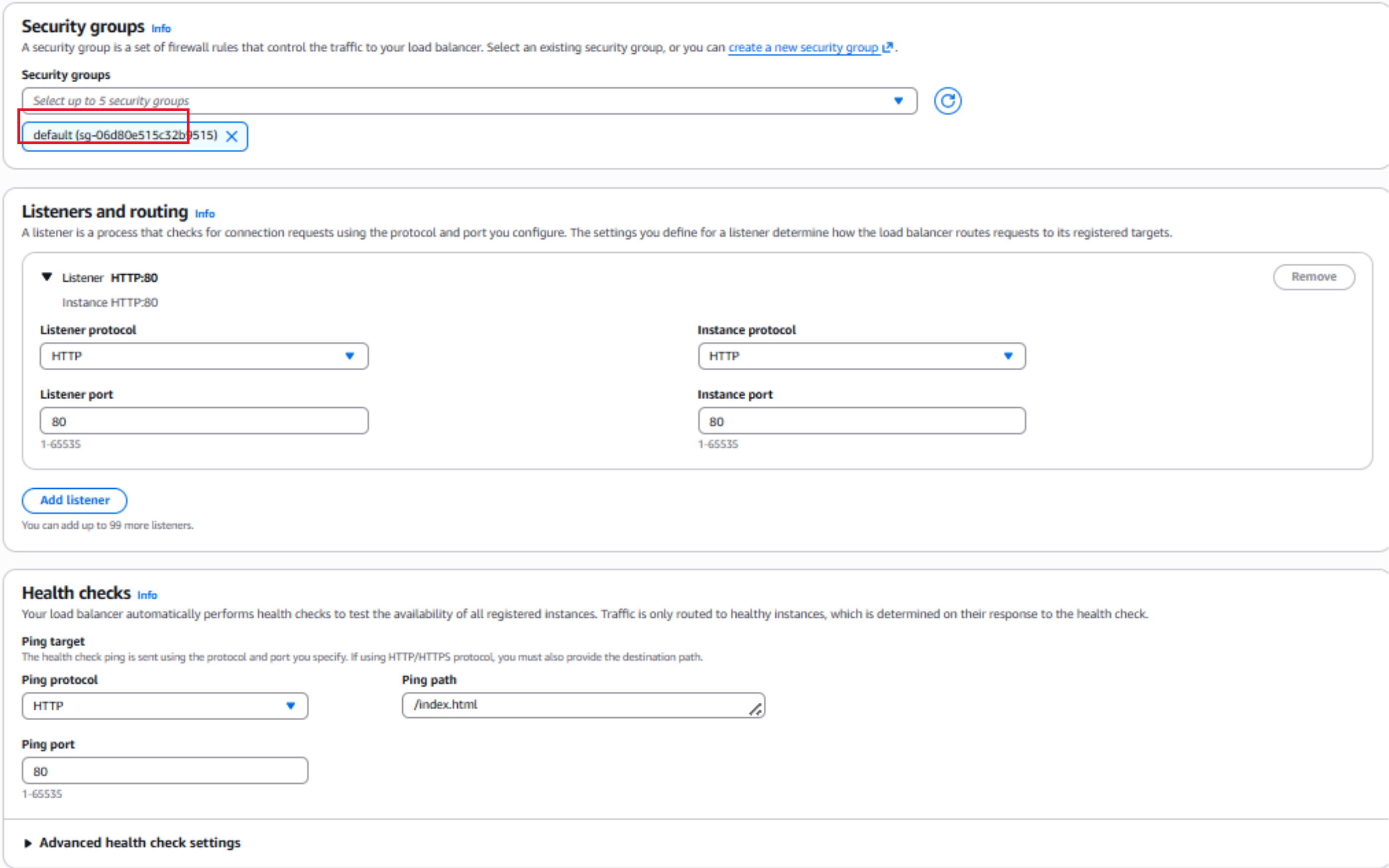
- Go to **EC2 → Load Balancers → Create Load Balancer → Classic Load Balancer**.



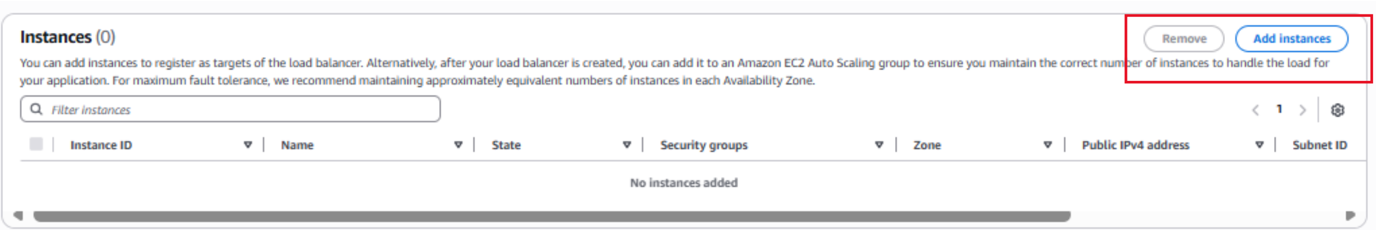
- Provide a name (e.g., Myclb). → choose the subnets.

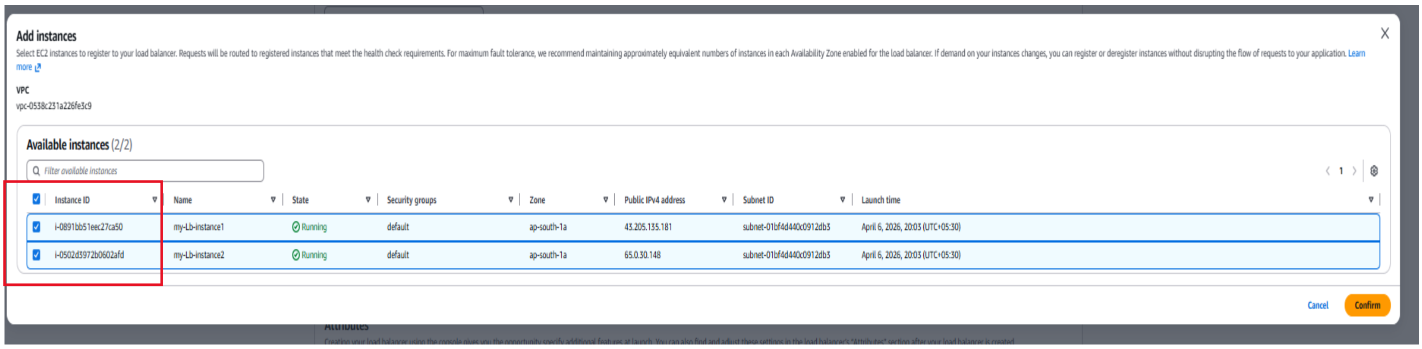


- Select **HTTP (port 80)** as the listener.
- Assign a security group that allows inbound HTTP.

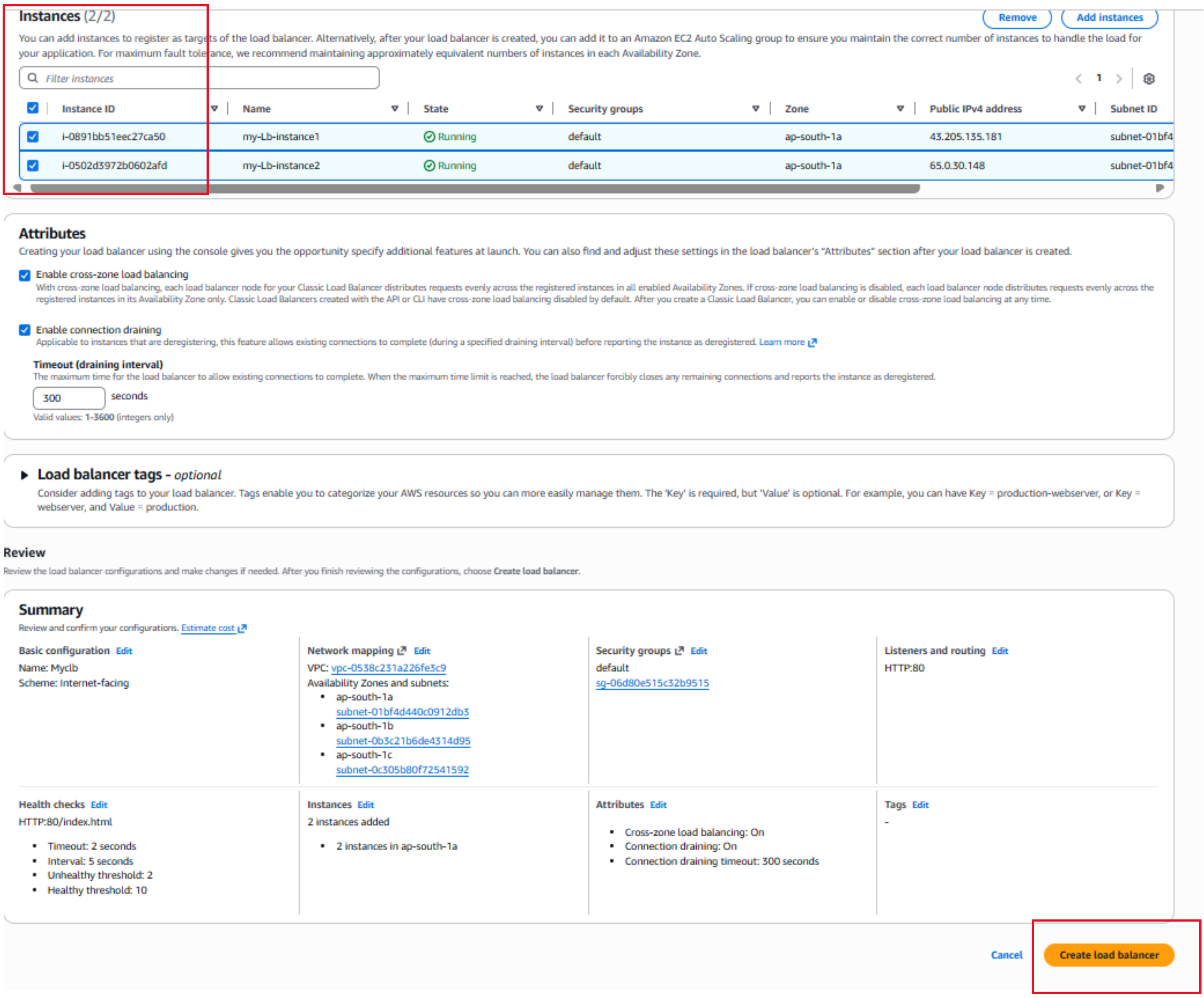


- Instance, add instance → select our instances → my-LB-instance1 & my-LB-instance2



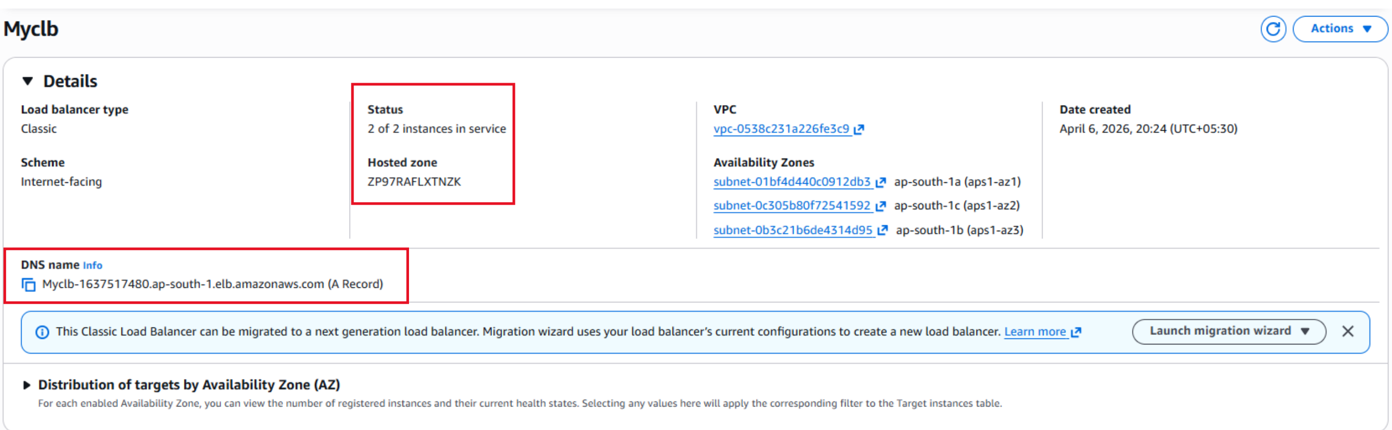


- Create Load Balancer.



Step no:05 Once the load balancer has been created, wait until the status of the instances in it become active.

Then copy the DNS paste it in a private tab.



We can see the output there, refresh the page we can see the traffic is now passed to the next instance. It shows load balancer is distributing the traffic across the two instances we have created.


welcome to ip-172-31-47-173.ap-s
Not secure mycIb-1637517480.ap-south-1.elb.amazonaws.com
NON PARAMETRIC...
EVS CHAP 1.pdf
Dell
View Ads | VuexyB...
Gmail
YouTube
Maps
New Tab
hello from ip-172-31-47-173.ap-south-1.compute.internal


welcome to ip-172-31-36-209.ap-s
Not secure mycIb-1637517480.ap-south-1.elb.amazonaws.com
NON PARAMETRIC...
EVS CHAP 1.pdf
Dell
View Ads | VuexyB...
Gmail
YouTube
Maps
New
hello from ip-172-31-36-209.ap-south-1.compute.internal